

Wizard Duel

Card game temático do universo Harry Potter, desenvolvido com Node.js + Express no back-end e HTML/CSS/JS puro no front-end, consumindo a [PotterDB API](#).

Como executar

```
npm install
npm start
# Acesse: http://localhost:3000
```

Como funciona o jogo

- Cada carta representa um personagem do universo Harry Potter com 4 atributos: **Poder**, **Magia**, **Defesa** e **Sorte**
- Na fase de draft, o jogador escolhe 2 personagens para compor seu deck
- A cada rodada, o jogador escolhe um feitiço para atacar o personagem CPU
- O personagem com HP zerado é eliminado; o dono de quem eliminar mais personagens vence o duelo

Trabalho Prático — Refatoração com ESLint (Airbnb)

Objetivo

O projeto foi desenvolvido com pressa e apresenta diversos problemas de qualidade de código. Sua tarefa é **refatorar o código sem alterar o comportamento visível da aplicação**, usando o ESLint com o guia de estilo da Airbnb como referência.

A aplicação deve continuar funcionando corretamente após a refatoração. Qualquer quebra de funcionalidade será penalizada.

Problemas a corrigir

Analise o código e identifique os problemas de qualidade, categorizados abaixo.

1. Números Mágicos

Valores literais sem contexto espalhados pelo código.

Exemplos: valores de atributos (`90` , `85` , `75`), tamanhos de pack/deck (`100` , `4` , `2` , `5`), duração de timeouts (`800` , `700`), número de feitiços (`20`).

2. Nomes Sem Significado

Variáveis e parâmetros com nomes que não comunicam intenção.

Exemplos: `d` , `r` , `tmp` , `c` , `a` , `obj` , `x` , `y` , `z` , `pw` , `mg` , `df` , `pg` .

3. Funções com Múltiplas Responsabilidades

Funções que fazem mais de uma coisa.

Exemplos: as rotas `/api/pack` e `/api/cpu-deck` fazem fetch, filtram, calculam atributos e embaralham tudo na mesma função.

4. Código Duplicado (DRY)

Trechos idênticos ou muito similares repetidos.

Exemplos: o cálculo de atributos (`pw` , `mg` , `df`) e a lógica de embaralhamento estão duplicados entre `/api/pack` e `/api/cpu-deck` .

5. Code Smells Gerais

- `var` ao invés de `const` / `let`
- Concatenação de strings ao invés de template literals
- Construção manual de HTML via concatenação de strings
- `console.log` para tratamento de erros
- `==` ao invés de `===`

Etapas da Refatoração

Renomeação de Variáveis e Funções

Renomeie todas as variáveis com nomes sem significado para nomes que expressem claramente sua intenção.

Extração de Constantes

Crie um arquivo `constants.js` e mova para ele todos os números mágicos.

Eliminação de Código Duplicado

O cálculo de atributos e o embaralhamento estão duplicados entre duas rotas. Extraia essa lógica para funções reutilizáveis.

Separação de Responsabilidades

Reorganize o projeto na seguinte estrutura:

```
wizard-duel/
├── index.js           ← apenas inicializa o servidor
├── constants.js       ← constantes da aplicação
├── routes/
│   ├── characters.js  ← rotas de personagens
│   ├── spells.js      ← rotas de feitiços
│   └── game.js        ← rotas do jogo
├── services/
│   ├── potterApi.js   ← comunicação com a PotterDB API
│   └── statsCalculator.js ← lógica de cálculo de atributos
└── public/
    ├── index.html
    ├── js/
    │   ├── game.js    ← lógica do jogo
    │   ├── render.js  ← funções de renderização
    │   └── api.js      ← chamadas ao back-end
    └── css/
        └── style.css   ← estilos separados do HTML
```

Penalidade 1: 0,1 ponto descontado por cada erro de lint reportado pelo ESLint na entrega final.

Penalidade 2: O trabalho será zerado caso a aplicação não execute ou apresente falhas decorrentes da refatoração.

Entrega

- Repositório Git com histórico de commits organizado (**um commit por etapa**)
- Arquivo REFATORACA0.md documentando:
 - Lista de problemas encontrados
 - Decisões tomadas durante a refatoração